[University Name]

Department of Computer Science

SkillBridge

A University Final Year Project

Submitted By

Author: [Author 1 Name]

Author: [Author 2 Name]

Supervised By

[Supervisor Name]

Submitted in partial fulfilment of the requirements for the degree of

Bachelor of Science in Computer Science

Academic Year: 2024–2025

March 19, 2026

Declaration

We hereby declare that this project report, titled **"SkillBridge"**, is our own original work carried out as a Final Year Project at **[University Name]**. This report has not been submitted, in whole or in part, for any other degree or examination at any university. All sources of information have been duly cited and acknowledged.

[Author 1 Name] _____

Signature

[Author 2 Name] _____

Signature

Date: March 19, 2026

Supervisor: [Supervisor Name] _____

Abstract

SkillBridge is a full-stack freelance marketplace platform developed as a Final Year Project. It bridges the gap between clients who have project requirements and skilled freelancers who can fulfil those requirements, operating within a university context to demonstrate the end-to-end design and implementation of a modern, production-grade web application.

The platform supports three distinct user roles — **Client**, **Freelancer**, and **Admin** — each with a bespoke dashboard, workflow, and set of features. Clients can post projects (with optional AI-assisted scoping), browse freelancer profiles, review proposals, manage contracts through milestone-based delivery, and participate in a blind peer-review system. Freelancers can onboard with a multi-step profile setup, browse and save project listings, submit proposals funded by the platform's internal *SkillToken* economy, track active contracts, and view earnings.

The system is built upon a modern technology stack: a **Node.js / Express.js** REST API written in **TypeScript**, a **MongoDB** database accessed via the **Prisma ORM**, **Redis** for caching and rate-limiting, **Cloudinary** for media storage, **Nodemailer** for transactional emails, and the **OpenAI API** for the AI project-scoping assistant. The frontend is a single-page application built with **React**, **Vite**, and **TypeScript**, featuring role-based route guards, a responsive design, dark/light theme support, and real-time-capable chat infrastructure.

Key contributions include: a token-economy incentive mechanism for proposal submission, an AI-powered project discovery assistant, a multi-step onboarding flow with profile completion scoring, a dispute resolution module with admin oversight, and a blind review system that reveals ratings only once both parties have submitted.

This report documents the complete requirements, design, architecture, implementation, and testing of the SkillBridge system across both the backend API and the React-based frontend application.

Keywords: Freelance Marketplace, REST API, Node.js, TypeScript, React, MongoDB, Prisma, Token Economy, AI Scoping, Role-Based Access Control.

Acknowledgements

We would like to express our sincere gratitude to our project supervisor, [**Supervisor Name**], for the invaluable guidance, constructive feedback, and continuous encouragement throughout the course of this project. Their expertise and insights were instrumental in shaping both the technical direction and academic quality of this work.

We extend our heartfelt thanks to the faculty and staff of the **Department of Computer Science** at [**University Name**] for equipping us with the knowledge and skills necessary to undertake a project of this complexity.

We are also grateful to our families and friends for their unwavering moral support and patience throughout the development of this project.

Finally, we acknowledge the open-source community whose tools, libraries, and frameworks — including Node.js, React, Prisma, and many others — made this project possible.

[Author 1 Name]
[Author 2 Name]

March 19, 2026

Contents

Declaration . . . . . **i**

Abstract . . . . . **ii**

Acknowledgements . . . . . **iii**

List of Figures . . . . . **ix**

List of Tables . . . . . **x**

List of Abbreviations . . . . . **xii**

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Problem Statement 1

1.3 Objectives 2

1.4 Scope 2

1.5 Report Organisation 3

2 Literature Review and Related Work **4**

2.1 Overview 4

2.2 Existing Freelance Marketplace Platforms 4

2.2.1 Upwork 4

2.2.2 Fiverr 4

2.2.3 Freelancer.com 5

2.2.4 Toptal 5

2.3 Comparative Analysis 5

2.4 Academic Research 5

2.4.1 Token Economies in Online Platforms 5

2.4.2 AI-Assisted Requirements Engineering 6

2.4.3 Blind Review Systems 6

2.4.4 REST API Design Principles 6

2.5 Identified Research Gaps 6

3 System Requirements **7**
3.1 Overview 7
3.2 Stakeholders 7
3.3 Functional Requirements 7
3.3.1 Authentication and User Management 7
3.3.2 Client Features 8
3.3.3 Freelancer Features 9
3.3.4 Admin Features 9
3.3.5 SkillToken Economy 10
3.3.6 AI Features 10
3.4 Non-Functional Requirements 11
3.5 Use Case Summary 12

4 System Architecture and Design **13**
4.1 Architectural Overview 13
4.2 Technology Stack 14
4.2.1 Backend 14
4.2.2 Frontend 15
4.3 Backend Application Structure 15
4.4 Request Lifecycle 16
4.5 Frontend Application Structure 17
4.6 Routing Architecture 18
4.7 State Management 18

5 Database Design **19**
5.1 Overview 19
5.2 Enumeration Types 19
5.3 Core Data Models 20
5.3.1 User 20
5.3.2 Session 21
5.3.3 ClientProfile and FreelancerProfile 21
5.3.4 Project 22
5.3.5 Proposal 22
5.3.6 Contract, Milestone, and Payment 22
5.3.7 Dispute 22
5.3.8 Review 22
5.3.9 TokenTransaction 23
5.3.10 Additional Models 23
5.4 Database Design Principles 23

6 Backend Implementation **25**
6.1 Overview 25
6.2 API Route Groups 25
6.3 Authentication Controller 26
6.4 Freelancer Onboarding Controller 27
6.5 Project Controller 27
6.6 Proposal Controller 28
6.7 Middleware Chain 28
6.7.1 Authentication Middleware 28
6.7.2 Role Middleware 28
6.7.3 Upload Middleware 29
6.7.4 Request Logger 29
6.8 Backend Directory Structure 29
6.9 Utility Modules 31
6.10 Browse Module 31
6.11 Cron Jobs 31

7 Frontend Implementation **33**
7.1 Overview 33
7.2 Application Entry Point 33
7.3 Authentication Context 33
7.4 Route Guards 34
7.5 Page Inventory 34
7.5.1 Public Pages 34
7.5.2 Authentication Pages 35
7.5.3 Client Pages 36
7.5.4 Freelancer Pages 37
7.5.5 Admin Pages 37
7.6 Frontend Directory Structure 38
7.7 Multi-Step Project Posting Form 38
7.8 Freelancer Onboarding Flow 38
7.9 Design System and Theming 39
7.10 State Management 39

8 SkillToken Economy **40**
8.1 Overview 40
8.2 Design Rationale 40
8.3 Token Events 41
8.4 Token Cost Calculation 41
8.5 Token Ledger Model 42

8.6 Frontend Token Page . . . 42
8.7 Admin Token Controls . . . 42

9 AI Integration . . . 44
9.1 Overview . . . 44
9.2 System Design . . . 44
9.2.1 Request Flow . . . 44
9.2.2 System Prompt Design . . . 45
9.3 AiScopingSession Model . . . 45
9.4 Frontend AI Assistant Page . . . 45
9.5 Error Handling and Edge Cases . . . 46
9.6 Use of GPT-4o Model . . . 46

10 Security and Authentication . . . 47
10.1 Overview . . . 47
10.2 Password Security . . . 47
10.3 JWT Authentication . . . 47
10.3.1 Token Rotation . . . 48
10.4 Google OAuth 2.0 . . . 48
10.5 OTP Email Verification . . . 49
10.6 Role-Based Access Control (RBAC) . . . 49
10.7 Environment Variable Security . . . 49
10.8 CORS Configuration . . . 50
10.9 File Upload Security . . . 50
10.10 XSS and Injection Prevention . . . 50
10.11 Account Security Features . . . 50

11 Testing and Validation . . . 52
11.1 Overview . . . 52
11.2 Testing Strategy . . . 52
11.3 API Testing . . . 52
11.3.1 Authentication Flow Tests . . . 53
11.3.2 Project and Proposal Tests . . . 53
11.3.3 Token Economy Tests . . . 54
11.4 Data Seeding . . . 54
11.5 Frontend Validation Testing . . . 55
11.6 TypeScript Type Safety . . . 55
11.7 Known Limitations . . . 55

12 Conclusion and Future Work . . . 57

12.1 Summary 57
12.2 Achievements 58
12.3 Limitations 58
12.4 Future Work 58
12.4.1 Technical Enhancements 59
12.4.2 Feature Enhancements 59
12.5 Conclusion 59

Bibliography **61**

A API Endpoint Reference **62**

B Environment Variables Reference **66**

C Database Schema Quick Reference **69**

List of Figures

4.1 High-Level Three-Tier Architecture of SkillBridge 13

List of Tables

2.1 Comparative Feature Analysis of Freelance Platforms 5

3.1 System Stakeholders 7
3.2 Non-Functional Requirements 11
3.3 Primary Use Cases 12

4.1 Backend Technology Stack 14
4.2 Frontend Technology Stack 15
4.3 Frontend State Management Strategy 18

5.1 Database Enumeration Types 20
5.2 User Model Fields 21
5.3 Additional Database Models 23

6.1 API Route Groups 26
6.2 Utility Module Responsibilities 31

7.1 Public-Facing Pages 34
7.2 Authentication Pages 35
7.3 Client Portal Pages 36
7.4 Freelancer Portal Pages 37
7.5 Admin Portal Pages 37
7.6 Frontend State Management Strategy 39

8.1 SkillToken Ledger Events 41
8.2 TokenTransaction Fields 42

9.1 AiScopingSession Model Fields 45

10.1 JWT Token Properties 48

11.1 Testing Strategy Overview 52
11.2 Authentication API Test Cases 53
11.3 Project and Proposal Test Cases 53
11.4 SkillToken Economy Test Cases 54

11.5 Database Seed Scripts 54

12.1 Project Objectives vs. Achievements 58

C.1 All Database Models Summary 70

List of Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
REST	Representational State Transfer
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JWT	JSON Web Token
OTP	One-Time Password
OAuth	Open Authorisation
RBAC	Role-Based Access Control
ORM	Object-Relational Mapper
DB	Database
SQL	Structured Query Language
NoSQL	Not Only SQL
SPA	Single Page Application
UI	User Interface
UX	User Experience
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
CDN	Content Delivery Network
AI	Artificial Intelligence
LLM	Large Language Model
CRUD	Create, Read, Update, Delete
ER	Entity-Relationship
FYP	Final Year Project
MVC	Model-View-Controller
TS	TypeScript
JS	JavaScript
npm	Node Package Manager
env	Environment (Variables)
ID	Identifier
URL	Uniform Resource Locator
UUID	Universally Unique Identifier

Abbreviation	**Full Form**
CORS	Cross-Origin Resource Sharing
CI	Continuous Integration

Chapter 1

Introduction

1.1 Background and Motivation

The global freelance economy has experienced exponential growth over the past decade. Platforms such as Upwork [1], Fiverr [2], Freelancer.com [3], and Toptal [4] have collectively processed billions of dollars in project payments, demonstrating the commercial viability and societal demand for online talent marketplaces. According to recent economic studies [5], the proportion of the workforce engaging in freelance or contract work has risen significantly, driven by globalisation of the talent pool, the widespread adoption of remote work, and the increasing demand for specialised, on-demand skills.

Despite the maturity of existing platforms, significant pain-points persist for both clients and freelancers. These include opaque matching algorithms, lack of verified skill credentials, exploitative commission models, one-sided review systems, and limited support for project scoping — particularly for non-technical clients who struggle to articulate requirements in a form legible to software developers.

SkillBridge was conceived to address these limitations within a controlled, demonstrable academic context. It is a full-stack freelance marketplace that integrates a custom *SkillToken* economy to incentivise quality participation, an AI-powered project scoping assistant to lower the barriers for non-technical clients, and a blind peer-review system that ensures rating integrity.

1.2 Problem Statement

The core problem this project addresses can be stated as follows:

> *Existing freelance marketplace platforms do not adequately (a) support non-technical clients in scoping and posting structured project requirements, (b) provide freelancers with transparent and gamified incentive mechanisms, or (c) ensure integrity in the mutual review process between clients and service providers.*

SkillBridge proposes an integrated solution encompassing all three dimensions through modern web technologies, a well-designed database schema, and machine-learning-assisted tooling.

1.3 Objectives

The primary objectives of this project are:

1. Design and develop a secure, multi-role RESTful API capable of managing user authentication, project lifecycle management, proposal submission, contract execution, and dispute resolution.
2. Build a responsive, role-aware single-page application providing distinct, intuitive dashboards for Clients, Freelancers, and Administrators.
3. Implement an internal **SkillToken** economy that controls proposal submission costs, awards participation bonuses, and maintains a full transaction ledger.
4. Integrate the **OpenAI API** to provide an AI project-scoping assistant that translates informal client descriptions into structured project specifications.
5. Design and implement a **blind review system** that reveals ratings only after both parties have submitted their reviews or a timeout has elapsed.
6. Implement **role-based access control (RBAC)** ensuring that all actions are authorised for the correct role.
7. Provide a **multi-step onboarding flow** for freelancers with profile completion scoring and draft-saving support.

1.4 Scope

The scope of SkillBridge encompasses:

- **Backend**: A Node.js/Express REST API written in TypeScript, backed by MongoDB via the Prisma ORM, with Redis-based caching and session management.
- **Frontend**: A React/Vite single-page application with full routing, context-based state management, and a Shadcn/Radix UI component library.
- **Authentication**: Email/password with OTP email verification, Google OAuth 2.0, JWT access/refresh token rotation, and password reset flows.
- **Core Features**: Project posting, proposal bidding, contract management, milestone tracking, payment status simulation, messaging, disputes, reviews, and

admin moderation.

- **Advanced Features**: AI project scoping, SkillToken economy, browse/search with interaction tracking, and freelancer gig listings.

Out of scope for this project are live payment gateway integration (simulated via status fields), mobile applications, and infrastructure deployment (the system is documented as a locally-run development setup).

1.5 Report Organisation

The remainder of this report is structured as follows:

Chapter 2 reviews related work and existing freelance marketplace platforms.

Chapter 3 presents the functional and non-functional requirements of the system.

Chapter 4 describes the overall system architecture, technology stack, and component design.

Chapter 5 documents the database design including all data models and their relationships.

Chapter 6 covers the backend API implementation in detail.

Chapter 7 covers the frontend React application implementation.

Chapter 8 explains the SkillToken economy design and implementation.

Chapter 9 documents the AI integration module.

Chapter 10 addresses security mechanisms, authentication, and authorisation.

Chapter 11 describes the testing approach and results.

Chapter 12 concludes with a summary, limitations, and future work.

Chapter 2

Literature Review and Related Work

2.1 Overview

This chapter analyses existing freelance marketplace platforms and academic research relevant to the design of SkillBridge. The review identifies the strengths and limitations of current systems, establishing the research gap that this project aims to address.

2.2 Existing Freelance Marketplace Platforms

2.2.1 Upwork

Upwork [1] is the world's largest freelance marketplace by revenue. It supports both hourly and fixed-price contracts and provides a comprehensive suite of tools including time-tracking, escrow payments, and a dispute resolution centre. Upwork's matching algorithm uses a proprietary "Job Success Score" (JSS) that combines client feedback, project outcomes, and account history.

Limitations: The platform charges up to 20% freelancer commission on earnings below $500, which many users consider excessive. The review system, while thorough, can be gamed by cancelling contracts before feedback is submitted. Non-technical clients still struggle with the proposal review process.

2.2.2 Fiverr

Fiverr [2] popularised the "gig" model, where freelancers create pre-packaged service offerings rather than bidding on client jobs. This model significantly lowers friction for clients who need a well-defined service.

Limitations: The gig model disadvantages custom, complex projects. Prices are anchored low, creating race-to-the-bottom dynamics. Review manipulation is a documented problem.

2.2.3 Freelancer.com

Freelancer.com [3] operates a competitive bidding model similar to SkillBridge. It features contests, milestone-based payments, and a large global user base.

Limitations: The platform is notorious for low-quality, low-cost proposals and a high percentage of spam bids. There is no AI-assisted scoping feature. Dispute resolution is slow and often unsatisfactory.

2.2.4 Toptal

Toptal [4] takes the opposite approach — an elite, invitation-only talent network where fewer than 3% of applicants are accepted. This ensures quality but sacrifices accessibility.

Limitations: Not suitable for small projects or emerging-market freelancers. Opaque vetting process.

2.3 Comparative Analysis

Table 2.1: Comparative Feature Analysis of Freelance Platforms

Feature	**Upwork**	**Fiverr**	**Freelancer**	**Toptal**	**SkillBridge**
AI Project Scoping	No	No	No	No	**Yes**
Token Economy	No	No	No	No	**Yes**
Blind Review System	No	No	No	No	**Yes**
Milestone Contracts	Yes	Yes	Yes	Yes	**Yes**
Dispute Resolution	Yes	Yes	Yes	Yes	**Yes**
Google OAuth	Yes	Yes	No	No	**Yes**
Admin Dashboard	N/A	N/A	N/A	N/A	**Yes**
Role-Based Access	Yes	Yes	Yes	Yes	**Yes**
Open-Source Stack	No	No	No	No	**Yes**

2.4 Academic Research

2.4.1 Token Economies in Online Platforms

The concept of internal token economies has been extensively studied in the context of online communities and incentive design. Token-based systems have been shown to increase user engagement, reduce spam, and reward high-quality contributions [6]. SkillBridge implements a *SkillToken* economy where tokens are spent to submit

proposals and awarded for registration and monthly activity, creating a self-regulating market for proposal quality.

2.4.2 AI-Assisted Requirements Engineering

The application of Large Language Models (LLMs) to requirements engineering is a growing research area. LLMs such as GPT-4 [7] have demonstrated the ability to transform informal natural-language descriptions into structured requirements artefacts. SkillBridge leverages this capability to help non-technical clients formulate project postings with appropriate category classifications, budget ranges, size estimates, and required skills.

2.4.3 Blind Review Systems

The "blind review" construct is borrowed from academic peer review, where reviewer identities are hidden to reduce bias. Applied to marketplace review systems, both-party-blind reviews — where ratings are only revealed once both parties have submitted — have been shown to produce more honest and balanced feedback.

2.4.4 REST API Design Principles

The architectural style of REST, originally defined by Fielding [8], underpins the entire backend API of SkillBridge. Key principles applied include stateless communication, resource-oriented URL design, standard HTTP method semantics, and JSON as the data interchange format.

2.5 Identified Research Gaps

Based on the above review, the following gaps justify the development of SkillBridge:

1. No major freelance platform provides an **AI-assisted project scoping tool** integrated into the project posting workflow.
2. No platform implements a **token-based proposal economy** to dissuade low-quality mass bidding.
3. No platform implements a **fully blind mutual review system** where both reviews are hidden until simultaneous reveal.
4. The academic community lacks a fully-documented, open-architecture reference implementation of a freelance marketplace for educational purposes.

SkillBridge directly addresses all four of these gaps.

Chapter 3

System Requirements

3.1 Overview

This chapter presents the complete set of functional and non-functional requirements for the SkillBridge system. Requirements were elicited through analysis of the problem domain, study of competing platforms, and iterative design sessions.

3.2 Stakeholders

Table 3.1: System Stakeholders

Stakeholder	Description
Client	A registered user who posts projects, reviews proposals, manages contracts, and leaves reviews.
Freelancer	A registered user who submits proposals, performs contracted work, and receives payment.
Admin	A platform operator who moderates content, resolves disputes, and manages platform settings.
System	Automated processes including cron jobs, email notifications, and the AI assistant.

3.3 Functional Requirements

3.3.1 Authentication and User Management

FR-AUTH-1 The system shall allow users to register with an email address and password.

FR-AUTH-2 The system shall send a One-Time Password (OTP) to the user's email upon registration for verification.

FR-AUTH-3 The system shall allow users to log in using verified email and password credentials.

FR-AUTH-4 The system shall support Google OAuth 2.0 for social login.

FR-AUTH-5 The system shall issue JWT access tokens (short-lived) and refresh tokens (long-lived) upon authentication.

FR-AUTH-6 The system shall rotate refresh tokens on each use, invalidating the previous token.

FR-AUTH-7 The system shall allow users to reset their password via an email-based secure token link.

FR-AUTH-8 The system shall allow users to select a role (CLIENT or FREELANCER) after initial registration.

FR-AUTH-9 The system shall enforce role-based access control on all protected routes.

FR-AUTH-10 The system shall allow admins to ban users and provide a ban reason.

FR-AUTH-11 The system shall allow users to log out, invalidating their current session.

3.3.2 Client Features

FR-CLIENT-1 The system shall allow clients to create, edit, save as draft, and publish project listings.

FR-CLIENT-2 A project listing shall include: title, description, category/subcategory, budget (fixed or hourly), size, deadline, required skills, experience level, language preference, location preference, hiring method, and optional attachments.

FR-CLIENT-3 The system shall allow clients to view all proposals submitted for their projects.

FR-CLIENT-4 The system shall allow clients to shortlist, accept, or reject proposals.

FR-CLIENT-5 The system shall allow clients to browse freelancer profiles with filtering by skills, experience level, rating, availability, and location.

FR-CLIENT-6 The system shall allow clients to invite specific freelancers to their projects.

FR-CLIENT-7 The system shall allow clients to manage active contracts, view milestones, and approve or request revision of submitted milestone work.

FR-CLIENT-8 The system shall allow clients to open a dispute for any active contract.

FR-CLIENT-9 The system shall allow clients to leave a review after contract completion.

FR-CLIENT-10 The system shall allow clients to access an AI assistant for project scoping.

FR-CLIENT-11 The system shall allow clients to manage their profile (company, industry, bio, location, preferences).

3.3.3 Freelancer Features

FR-FRLNC-1 The system shall provide a multi-step onboarding flow for freelancers to complete their profile.

FR-FRLNC-2 Onboarding steps shall include: basic info, skills, portfolio, certificates, education, availability, social links, and verification document upload.

FR-FRLNC-3 The system shall calculate a profile completion percentage as a ratio of filled mandatory fields.

FR-FRLNC-4 The system shall allow freelancers to save an onboarding draft without completing all mandatory fields.

FR-FRLNC-5 The system shall allow freelancers to browse open project listings with filtering and search.

FR-FRLNC-6 The system shall allow freelancers to save/bookmark projects of interest.

FR-FRLNC-7 The system shall allow freelancers to submit a proposal (cover letter, price, delivery time) to a project, deducting SkillTokens.

FR-FRLNC-8 The system shall allow freelancers to withdraw a pending proposal, refunding SkillTokens.

FR-FRLNC-9 The system shall allow freelancers to view all active contracts and submit milestone deliveries.

FR-FRLNC-10 The system shall allow freelancers to view their SkillToken balance and full transaction history.

FR-FRLNC-11 The system shall allow freelancers to create Gig listings (pre-packaged services).

FR-FRLNC-12 The system shall allow freelancers to leave a review after contract completion.

3.3.4 Admin Features

FR-ADMIN-1 The system shall provide an admin dashboard with platform analytics.

FR-ADMIN-2 The system shall allow admins to view, search, and ban/unban users.

FR-ADMIN-3 The system shall allow admins to moderate project listings (view, flag, remove).

FR-ADMIN-4 The system shall allow admins to manage disputes: assign, review, resolve with a resolution note and decision.

FR-ADMIN-5 The system shall allow admins to grant or deduct SkillTokens from any freelancer.

FR-ADMIN-6 The system shall log all admin actions with the target entity, action type, and timestamp.

3.3.5 SkillToken Economy

FR-TOKEN-1 Each new freelancer shall receive a registration bonus in SkillTokens.

FR-TOKEN-2 Submitting a proposal shall deduct a fixed quantity of SkillTokens from the freelancer's balance.

FR-TOKEN-3 Withdrawing a pending proposal shall refund the SkillTokens.

FR-TOKEN-4 A cron job shall award monthly SkillTokens to active freelancers.

FR-TOKEN-5 Every token transaction shall be recorded in an immutable ledger with the balance snapshot.

3.3.6 AI Features

FR-AI-1 The system shall provide an AI chat assistant accessible from the client dashboard.

FR-AI-2 The client shall be able to input an informal project description in natural language.

FR-AI-3 The AI assistant shall return a structured response including: recommended category/subcategory, estimated budget range, recommended project size, and a list of suggested skills.

FR-AI-4 The AI scoping session shall be stored in the database and optionally linked to the resulting project.

3.4 Non-Functional Requirements

Table 3.2: Non-Functional Requirements

ID	Category	Requirement
NFR-01	Security	All passwords shall be hashed using bcrypt before storage.
NFR-02	Security	All API routes (except public) shall require a valid JWT.
NFR-03	Security	Sensitive environment variables (keys, secrets) shall never be committed to version control.
NFR-04	Performance	API endpoints shall respond within 500ms for standard CRUD operations under normal load.
NFR-05	Performance	Redis shall be used to cache frequently accessed, rarely mutated data.
NFR-06	Scalability	The database design shall support horizontal scaling via MongoDB Atlas sharding.
NFR-07	Usability	The frontend shall be fully responsive and functional on viewports from 375px to 1920px wide.
NFR-08	Usability	The frontend shall support both light and dark themes.
NFR-09	Reliability	File uploads shall be stored in Cloudinary CDN, not on the application server filesystem.
NFR-10	Maintainability	All source code shall be written in TypeScript with strict type checking enabled.
NFR-11	Maintainability	The database schema shall be managed via Prisma migrations.
NFR-12	Availability	Transactional emails shall be delivered via Nodemailer with a retry mechanism.

3.5 Use Case Summary

Table 3.3: Primary Use Cases

UC-ID	Actor	Use Case	Brief Description
UC-01	Client	Post Project	Client creates and publishes a project listing.
UC-02	Client	Review Proposals	Client views, shortlists, accepts proposals.
UC-03	Client	Manage Contract	Client tracks milestones and approves deliveries.
UC-04	Client	AI Scope Project	Client describes project; AI generates spec.
UC-05	Client	Browse Freelancers	Client filters and views freelancer profiles.
UC-06	Freelancer	Complete Onboarding	Freelancer fills multi-step profile form.
UC-07	Freelancer	Submit Proposal	Freelancer bids on a project, spending tokens.
UC-08	Freelancer	Submit Milestone	Freelancer submits deliverable with attachments.
UC-09	Freelancer	View Tokens	Freelancer views balance and transaction history.
UC-10	Admin	Resolve Dispute	Admin reviews evidence and issues resolution.
UC-11	Both	Leave Review	Both parties submit post-contract blind reviews.
UC-12	Both	Message via Chat	Parties communicate through contract chat room.

Chapter 4

System Architecture and Design

4.1 Architectural Overview

SkillBridge follows a **three-tier client-server architecture** with a clear separation of concerns between the presentation, application logic, and data tiers. The system is designed as a RESTful API backend consumed by a React single-page application frontend.

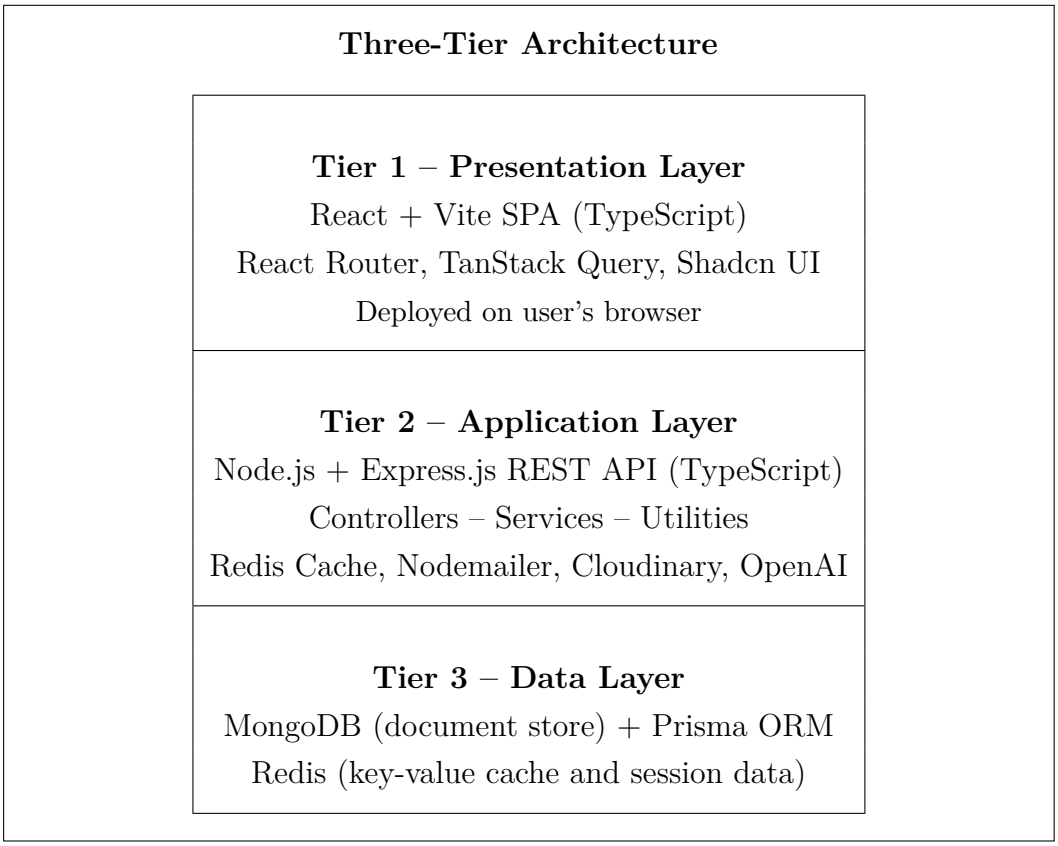

Figure 4.1: High-Level Three-Tier Architecture of SkillBridge

4.2 Technology Stack

4.2.1 Backend

Table 4.1: Backend Technology Stack

Technology	Version	Purpose
Node.js	20.x	JavaScript/TypeScript server runtime.
Express.js	5.x	HTTP web framework providing routing, middleware, and request handling.
TypeScript	5.x	Typed superset of JavaScript providing compile-time safety.
Prisma ORM	6.x	Type-safe ORM for database access and schema management.
MongoDB	7.x	NoSQL document database for flexible, scalable data storage.
Redis	7.x	In-memory data store for caching, rate-limiting, and OTP storage.
Cloudinary	2.x	Cloud-based image and file storage with CDN delivery.
Nodemailer	8.x	SMTP email sending for OTP, password reset, and notifications.
OpenAI	6.x	GPT API integration for the AI project scoping assistant.
Passport.js	0.7	Authentication middleware for Google OAuth 2.0.
bcrypt.js	3.x	Password hashing library.
jsonwebtoken	9.x	JWT signing and verification.
Zod	4.x	Runtime schema validation for request bodies.
Multer	2.x	Multipart form-data middleware for file uploads.
node-cron	4.x	Monthly SkillToken award cron job scheduling.

4.2.2 Frontend

Table 4.2: Frontend Technology Stack

Technology	Version	Purpose
React	18.x	Component-based UI framework.
Vite	5.x	Next-generation frontend build tool and dev server.
TypeScript	5.x	Static typing for the frontend codebase.
React Router	6.x	Client-side routing and navigation.
TanStack Query	5.x	Server state management, caching, and synchronisation.
Axios	1.x	HTTP client for API communication.
Shadcn/Radix UI	–	Accessible, composable UI component primitives.
Tailwind CSS	3.x	Utility-first CSS framework for styling.
Lucide React	–	Icon library.
React Hook Form	–	Performant, accessible form management.

4.3 Backend Application Structure

The backend follows a layered architecture within a monorepo structure. The `src/` directory is organised as follows:

```
src/
+-- app.ts                # Express app: middleware + route registration
+-- server.ts             # HTTP server bootstrap + port binding
+-- config/               # Configuration modules
|   +-- cloudinary.ts     # Cloudinary SDK init
|   +-- passport.ts       # Google OAuth strategy definition
|   +-- prisma.ts         # Prisma client singleton
|   \-- redis.ts          # Redis client configuration
+-- controllers/          # Route handler functions (thin layer)
|   +-- auth.controller.ts
|   +-- freelancer-onboarding.controller.ts
|   +-- freelancer.controller.ts
|   +-- project.controller.ts
|   +-- proposal.controller.ts
|   +-- token.controller.ts
|   +-- tracking.controller.ts
|   +-- contract.controller.ts
|   \-- metadata.controller.ts
+-- routes/               # Express Router definitions
|   +-- auth.routes.ts
```

```
|   +-- freelancer.routes.ts
|   +-- project.routes.ts
|   +-- proposal.routes.ts
|   +-- token.routes.ts
|   +-- tracking.routes.ts
|   +-- category.routes.ts
|   +-- metadata.routes.ts
|   \-- ai.routes.ts
+-- middlewares/         # Express middleware functions
|   +-- auth.middleware.ts   # JWT verification
|   +-- role.ts              # Role-based access guard
|   +-- upload.middleware.ts # Multer file handling
|   \-- logger.middleware.ts # Request logging
+-- services/
|   +-- token.service.ts
|   \-- tracking.service.ts
+-- modules/
|   \-- browse/          # Browse and discovery module
+-- utils/
|   +-- email.ts             # Email templates
|   +-- jwt.ts               # Token helpers
|   +-- redis.ts             # Redis wrappers
|   +-- uploadToCloudinary.ts
|   +-- profileCompletion.ts
|   +-- tokenCalculator.ts
|   \-- validators.ts
\-- ai/                  # AI assistant module
```

Listing 4.1: Backend Directory Structure

4.4 Request Lifecycle

Every HTTP request to the SkillBridge API follows a deterministic lifecycle:

1. **CORS Check**: The CORS middleware validates the `Origin` header against the allowed frontend list.
2. **Body Parsing**: `express.json()` and `cookie-parser` decode the request payload and cookies.
3. **Request Logging**: The custom `requestLogger` middleware records method, path, and response time.
4. **Authentication**: The `protect` middleware verifies the JWT access token from the request header and attaches the decoded user payload to `req.user`.

5. **Authorisation**: Role-specific middleware (e.g., `requireRole('CLIENT')`) checks that the authenticated user has the required role.
6. **Controller**: The matching route handler (controller function) is called. Zod schema validation may occur here.
7. **Service/ORM**: The controller calls Prisma to query or mutate the database, or delegates to a service function.
8. **Response**: The controller sends a JSON response with the appropriate HTTP status code.

4.5 Frontend Application Structure

The React frontend is organised by feature and role:

```
src/
+-- App.tsx              # Root component: providers + routing tree
+-- main.tsx             # Vite entry point
+-- contexts/
|   +-- AuthContext.tsx  # Global auth state (user, token, logout)
|   \-- ThemeContext.tsx # Light/dark theme toggle
+-- components/
|   +-- ProtectedRoute.tsx   # Auth + role guard components
|   +-- layout/              # Navbar, sidebar, footer
|   +-- common/              # Shared components (cards, badges)
|   +-- browse/              # Browse-specific components
|   +-- client/              # Client-specific components
|   +-- modals/              # Modal dialogs
|   \-- ui/                  # Shadcn UI primitives
+-- pages/
|   +-- LandingPage.tsx
|   +-- AboutPage.tsx
|   +-- HowItWorksPage.tsx
|   +-- auth/               # Login, Signup, OTP, OAuth, Reset
|   +-- client/             # 14 client-side pages
|   +-- freelancer/         # 12 freelancer-side pages
|   +-- admin/              # 6 admin pages
|   \-- legal/              # Privacy policy, Terms of service
+-- hooks/                  # Custom React hooks
\-- lib/                    # Utility functions, API client config
```

Listing 4.2: Frontend Directory Structure

4.6 Routing Architecture

The frontend uses React Router v6 with a nested route structure implementing three route guard types:

- **Public Routes**: Accessible without authentication (Landing, About, Login, Signup, etc.).
- **Guest Routes**: Only accessible when *not* authenticated; redirect logged-in users to their dashboard.
- **Protected Routes**: Require authentication. Further subdivided by `RoleRoute` into CLIENT-only, FREELANCER-only, and ADMIN-only sub-trees.

4.7 State Management

Table 4.3: Frontend State Management Strategy

State Type	Mechanism
Authentication state	React Context (`AuthContext`) — persisted in memory + HTTP-only cookie for refresh token.
Server state (API data)	TanStack Query — with automatic caching, refetching, and invalidation.
Form state	React Hook Form — with Zod schema validation.
Theme preference	React Context (`ThemeContext`) — persisted in `localStorage`.
UI state (modals, etc.)	Local `useState` within components.

Chapter 5

Database Design

5.1 Overview

SkillBridge uses **MongoDB** as the underlying database, accessed through the **Prisma ORM** (version 6.x). MongoDB's document model was chosen for its schema flexibility, horizontal scalability, and native JSON compatibility with Node.js. Prisma provides a type-safe, auto-generated client and a declarative schema language that makes the data model self-documenting.

The complete schema is defined in `prisma/schema.prisma` and comprises approximately **25 models** and **12 enumerations**. All primary keys are MongoDB ObjectIds mapped to Prisma's `@db.ObjectId` type.

5.2 Enumeration Types

The schema defines the following enumerations to enforce valid state values across models:

Table 5.1: Database Enumeration Types

Enum	Values
Role	CLIENT, FREELANCER, ADMIN
ProjectStatus	DRAFT, OPEN, IN_PROGRESS, UNDER_REVIEW, REVISION_REQUESTED, COMPLETED, CANCELLED, DISPUTED
ProjectSize	SMALL, MEDIUM, LARGE
ProposalStatus	PENDING, SHORTLISTED, ACCEPTED, REJECTED, WITHDRAWN
ContractStatus	ACTIVE, COMPLETED, CANCELLED, DISPUTED
MilestoneStatus	PENDING, IN_PROGRESS, SUBMITTED, APPROVED, REJECTED
PaymentStatus	PENDING, HELD_IN_ESCROW, RELEASED, REFUNDED, FAILED
DisputeStatus	OPEN, UNDER_REVIEW, RESOLVED, CLOSED
DisputeResolution	FAVOR_CLIENT, FAVOR_FREELANCER, SPLIT, DISMISSED
TokenWalletTxType	CREDIT, DEBIT
TokenTxReason	REGISTRATION_BONUS, MONTHLY_AWARD, PROPOSAL_SUBMITTED, PROPOSAL_WITHDRAWN, ADMIN_GRANT, ADMIN_DEDUCT
ExperienceLevel	ENTRY, MID, SENIOR, EXPERT
AvailabilityStatus	AVAILABLE, BUSY, UNAVAILABLE
InvitationStatus	PENDING, ACCEPTED, REJECTED, EXPIRED
MessageType	TEXT, FILE, SYSTEM
NotificationType	PROPOSAL_RECEIVED, PROPOSAL_ACCEPTED, PAYMENT_RELEASED, TOKEN_CREDITED, and others (16 total)

5.3 Core Data Models

5.3.1 User

The `User` model is the unified authentication entity. A single `users` collection stores all account types; the `role` field determines whether the user is a Client, Freelancer, or Admin, and controls which sub-profile document is created.

Table 5.2: User Model Fields

Field	Type	Description
`id`	ObjectId	Primary key.
`name`	String	Display name.
`email`	String	Unique email address.
`passwordHash`	String?	Bcrypt hash; null for OAuth-only accounts.
`googleId`	String?	Google OAuth sub (subject) identifier.
`role`	Role?	CLIENT, FREELANCER, or ADMIN.
`isEmailVerified`	Boolean	True after OTP verification.
`isIdVerified`	Boolean	True after admin verifies the uploaded ID.
`idDocumentUrl`	String?	Cloudinary URL of the uploaded identity document.
`isBanned`	Boolean	Set by admin; blocks login.
`profileImage`	String?	Cloudinary URL of avatar.
`createdAt`	DateTime	Account creation timestamp.
`lastActiveAt`	DateTime?	Used by the browse scoring algorithm.

5.3.2 Session

The `Session` model stores issued refresh tokens, enabling server-side revocation and preventing token reuse attacks.

5.3.3 ClientProfile and FreelancerProfile

Each role has a dedicated profile document (stored in `client_profiles` and `freelancer_profiles` collections) linked via a `userId` foreign reference. This one-to-one relationship keeps the core `User` document lean while allowing role-specific fields.

Key `FreelancerProfile` fields include:

- `skillTokenBalance` — current SkillToken balance.
- `profileCompletion` — integer 0–100 representing onboarding completeness.
- `experienceLevel` — `ExperienceLevel` enum.
- `availability` — `AvailabilityStatus` enum.
- `preferredCategories` — array of category slugs for browse personalisation.
- `preferredBudgetMin / Max` — browse filter defaults.

5.3.4 Project

The `Project` model represents a client's work posting. Key design decisions include:

- `status` follows the `ProjectStatus` state machine from `DRAFT` through `COMPLETED` or `CANCELLED`.
- `budgetType` can be `"fixed"` or `"hourly"`.
- `isAiScoped` and `aiScopeSummary` record whether the OpenAI assistant was used to generate the project spec.
- `attachments` is a `String[]` array of Cloudinary URLs.

5.3.5 Proposal

The `Proposal` model captures a freelancer's bid on a project. The `tokenCost` field records the SkillTokens deducted at submission. A unique constraint on `[projectId, freelancerProfileId]` enforces one proposal per freelancer per project.

5.3.6 Contract, Milestone, and Payment

Once a proposal is accepted, a `Contract` document is created. Each contract has an array of `Milestone` sub-documents representing deliverable checkpoints. Each milestone has an associated `Payment` document tracking the escrow and release status. The relationship chain is:

$$\texttt{Project} \rightarrow \texttt{Contract} \rightarrow \texttt{Milestone[]} \rightarrow \texttt{Payment}$$

5.3.7 Dispute

The `Dispute` model links a project to the client, freelancer, and optionally an admin. The `resolution` field uses the `DisputeResolution` enum to record the outcome (`FAVOR_CLIENT`, `FAVOR_FREELANCER`, `SPLIT`, or `DISMISSED`).

5.3.8 Review

The `Review` model implements the blind review system. The `isRevealed` boolean (default `false`) controls visibility. Reviews are revealed when both parties have submitted or a timeout elapses. A unique constraint on `[contractId, giverId]` prevents duplicate reviews.

5.3.9 TokenTransaction

An immutable ledger of all SkillToken movements. Every row stores the `amount`, `type` (CREDIT/DEBIT), `reason` (from `TokenTxReason` enum), and a `balanceAfter` snapshot. The snapshot enables full balance reconstruction and audit trail.

5.3.10 Additional Models

Table 5.3: Additional Database Models

Model	Purpose
`Skill`	Global skills catalogue (name, category).
`FreelancerSkill`	Many-to-many join: freelancer ↔ skill (with proficiency 1–5).
`ProjectSkill`	Many-to-many join: project ↔ required skill.
`PortfolioItem`	Freelancer's past work showcase items.
`Certificate`	Freelancer's professional certificates.
`Education`	Freelancer's education history.
`Gig`	Freelancer's pre-packaged service offering.
`Category`	Project category (e.g., Web Development).
`SubCategory`	Project sub-category (e.g., Frontend).
`Invitation`	Client-to-freelancer project invitation.
`ChatRoom`	One chat room per contract or pre-contract.
`Message`	Individual message within a chat room.
`Notification`	In-app notification for any event.
`AdminLog`	Immutable audit trail of admin actions.
`PlatformSetting`	Key-value store for global platform config.
`PasswordResetToken`	Secure time-limited token for password resets.
`AiScopingSession`	Stored AI assistant conversation for each scoping interaction.
`SavedProject`	Freelancer bookmarks for projects.
`BrowseInteraction`	Tracks freelancer project views for personalisation.

5.4 Database Design Principles

1. **Cascade deletes**: All child documents use `onDelete: Cascade` so that deleting a user removes all associated profiles, projects, proposals, and messages automatically.

2. **Unique constraints**: Applied on email (User), refresh tokens (Session), and join-table composite keys to prevent data duplication.

3. **Immutability for ledgers**: `TokenTransaction` and `AdminLog` records are never updated or deleted, ensuring a full audit trail.
4. **Snapshot fields**: `TokenTransaction.balanceAfter` stores the running balance at transaction time, enabling O(1) current-balance lookup.
5. **Denormalised counters**: `Project.proposalCount` and `Project.viewCount` are maintained as denormalised integers for fast read-path performance rather than computing counts via aggregation on every request.

Chapter 6

Backend Implementation

6.1 Overview

The SkillBridge backend is a Node.js/Express REST API written entirely in TypeScript. The entry point is `src/server.ts`, which bootstraps the HTTP server on a configurable port. The Express application is assembled in `src/app.ts`, where all global middleware and route groups are registered.

6.2 API Route Groups

The API is versioned under the `/api` prefix. Table 6.1 summarises all registered route groups.

Table 6.1: API Route Groups

Prefix	File	Responsibility
`/api/auth`	`auth.routes.ts`	Registration, OTP, login, logout, OAuth, password reset.
`/api/projects`	`project.routes.ts`	CRUD for project listings; status transitions.
`/api/freelancers`	`freelancer.routes.ts`	Freelancer onboarding, profile management, gigs.
`/api/proposals`	`proposal.routes.ts`	Submit, withdraw, shortlist, accept, reject proposals.
`/api/browse`	`browse.routes.ts`	Search/filter freelancers and projects with scoring.
`/api/track`	`tracking.routes.ts`	Record browse interactions, save/unsave projects.
`/api/tokens`	`token.routes.ts`	Token balance, history, admin grant/deduct.
`/api/categories`	`category.routes.ts`	Read-only category and subcategory lists.
`/api/metadata`	`metadata.routes.ts`	Languages and locations reference data.
`/api/ai`	`ai.routes.ts`	AI project scoping assistant endpoint.

6.3 Authentication Controller

The `auth.controller.ts` file implements the following endpoints:

`POST /api/auth/signup` Accepts `{name, email, password}`. Hashes the password with bcrypt, creates a `User` document with `isEmailVerified: false`, generates a 6-digit OTP, stores it in Redis with a 10-minute TTL, and sends it via Nodemailer.

`POST /api/auth/verify-otp` Accepts `{email, otp}`. Looks up the OTP in Redis, compares it, sets `isEmailVerified: true`, deletes the OTP key, and returns JWT tokens.

`POST /api/auth/login` Validates credentials, checks for ban status, generates an access token and refresh token, stores the refresh token as an HttpOnly cookie in a `Session` document.

POST /api/auth/refresh Reads the refresh token from the HttpOnly cookie, validates it against the database, rotates the token (delete old, issue new), and returns a new access token.

POST /api/auth/logout Deletes the session document and clears the cookie.

POST /api/auth/forgot-password Generates a UUID-based reset token stored in `PasswordResetToken`, sends an email with a link containing the token. Token expires in 1 hour.

POST /api/auth/reset-password Validates the reset token, hashes and updates the password, marks the token as used.

GET /api/auth/google Redirects to Google's OAuth consent screen via Passport.js.

GET /api/auth/google/callback Handles the OAuth callback. Uses Passport's Google strategy to find or create a `User`, then issues JWT tokens and redirects to the frontend.

6.4 Freelancer Onboarding Controller

`freelancer-onboarding.controller.ts` handles the multi-step onboarding flow. Key design points:

- All onboarding steps are `PATCH` requests to step-specific endpoints (e.g., `/api/freelancers/onbo` `/api/freelancers/onboarding/skills`).
- After each step, the `profileCompletion` score is recalculated using the `calculateProfileCompl` utility, which evaluates the number of filled mandatory fields as a fraction of total mandatory fields, multiplied by 100.
- A "Save Draft" mode accepts partial data without triggering mandatory field validation.
- File uploads (ID document, portfolio images) are handled by Multer in memory storage mode, then streamed to Cloudinary via `streamifier`.

6.5 Project Controller

`project.controller.ts` manages the full project lifecycle:

```
router.post('/', protect, requireRole('CLIENT'), createProject);
router.get('/', getAllProjects);
router.get('/:id', getProjectById);
router.patch('/:id', protect, requireRole('CLIENT'), updateProject)
    ;
```

```
router.delete('/:id', protect, requireRole('CLIENT'), deleteProject);
router.patch('/:id/publish', protect, requireRole('CLIENT'), publishProject);
```

Listing 6.1: Project Routes Example

The `createProject` handler accepts the full project specification, creates a `Project` document in `DRAFT` status, and optionally links `ProjectSkill` join documents for each required skill. The `publishProject` handler transitions status from `DRAFT` to `OPEN`.

6.6 Proposal Controller

The proposal workflow enforces the SkillToken economy:

1. **Submit**: Validate the freelancer has sufficient token balance. Deduct tokens atomically. Create `Proposal` document in `PENDING` status. Create a `TokenTransaction` record.
2. **Withdraw**: Only allowed if `status === PENDING`. Refund the original `tokenCost`. Update proposal status to `WITHDRAWN`. Create a `CREDIT TokenTransaction`.
3. **Accept**: Transition proposal to `ACCEPTED`, create a `Contract`, transition project to `IN_PROGRESS`, reject all other proposals.

6.7 Middleware Chain

6.7.1 Authentication Middleware

```
export const protect = async (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ message: 'Unauthorised' });
  const decoded = verifyAccessToken(token);
  req.user = decoded;
  next();
};
```

Listing 6.2: JWT Authentication Middleware (simplified)

6.7.2 Role Middleware

```
export const requireRole = (role: Role) => (req, res, next) => {
  if (req.user?.role !== role)
```

```
    return res.status(403).json({ message: 'Forbidden' });
  next();
};
```

Listing 6.3: Role Guard Middleware

6.7.3 Upload Middleware

The `upload.middleware.ts` configures Multer with in-memory storage (`memoryStorage()`) and a 10MB file size limit. Files are stored as `Buffer` objects on `req.file`, which are then piped to Cloudinary via `streamifier.createReadStream()`.

6.7.4 Request Logger

The custom `requestLogger` middleware logs the HTTP method, URL, response status code, and response time in milliseconds to stdout for every request, aiding development debugging.

6.8 Backend Directory Structure

```
src/
  app.ts                -- Express app: middleware + route
   registration
  server.ts             -- HTTP server bootstrap + port binding
  config/
    cloudinary.ts       -- Cloudinary SDK init
    passport.ts         -- Google OAuth strategy definition
    prisma.ts           -- Prisma client singleton
    redis.ts            -- Redis client configuration
  controllers/          -- Route handler functions (thin layer)
    auth.controller.ts
    freelancer-onboarding.controller.ts
    freelancer.controller.ts
    project.controller.ts
    proposal.controller.ts
    token.controller.ts
    tracking.controller.ts
    contract.controller.ts
    metadata.controller.ts
  routes/               -- Express Router definitions
    auth.routes.ts
    freelancer.routes.ts
    project.routes.ts
    proposal.routes.ts
```

```
    token.routes.ts
    tracking.routes.ts
    category.routes.ts
    metadata.routes.ts
    ai.routes.ts
  middlewares/
    auth.middleware.ts      -- JWT verification
    role.ts                 -- Role-based access guard
    upload.middleware.ts    -- Multer file handling
    logger.middleware.ts    -- Request logging
  services/
    token.service.ts
    tracking.service.ts
  modules/
    browse/             -- Browse & discovery module
  utils/
    email.ts            -- Email templates
    jwt.ts              -- Token signing/verification helpers
    redis.ts            -- Redis helper wrappers
    uploadToCloudinary.ts
    profileCompletion.ts
    tokenCalculator.ts
    validators.ts
  ai/                   -- AI assistant module
```

Listing 6.4: Backend Directory Structure (ASCII)

6.9 Utility Modules

Table 6.2: Utility Module Responsibilities

Utility	Responsibility
`jwt.ts`	Wraps `jsonwebtoken` sign/verify. Access token: 15 min. Refresh token: 7 days.
`email.ts`	HTML email templates for OTP, password reset, welcome, and notification emails.
`redis.ts`	Helper wrappers: `setEx`, `get`, `del` for Redis OTP and cache storage.
`uploadToCloudinary.ts`	Uploads a Buffer to a specified Cloudinary folder and returns the secure URL.
`profileCompletion.ts`	Counts non-null mandatory fields to compute the 0–100 completion score.
`tokenCalculator.ts`	Determines the SkillToken cost for a proposal based on project budget tiers.
`validators.ts`	Zod schemas for request body validation across multiple controllers.

6.10 Browse Module

The `modules/browse/` directory contains a self-contained browse and discovery feature. It implements:

- **Freelancer search**: Filter by skills, experience level, availability, minimum rating, hourly rate range, and location. Results are sorted by a composite *relevance score* incorporating profile completeness, rating, dispute ratio, and recent login activity.
- **Project browse**: Filter by category, budget range, project size, experience level, and skills. Records are sorted by recency and proposal count.
- **Interaction tracking**: `BrowseInteraction` records are created when a freelancer views a project, feeding back into personalised recommendations.

6.11 Cron Jobs

A monthly cron job (scheduled via `node-cron`) awards SkillTokens to qualifying active freelancers. A freelancer qualifies if they have been active (`lastLoginAt` within

the last 30 days) and have a validated profile. The job creates a `MONTHLY_AWARD` `TokenTransaction` for each qualifying freelancer and updates their `skillTokenBalance`.

Chapter 7

Frontend Implementation

7.1 Overview

The SkillBridge frontend is a single-page application (SPA) built with **React 18**, **Vite 5**, and **TypeScript**. The application is structured around role-based routing, server-state management via TanStack Query, and a consistent design system built on Shadcn UI / Radix UI primitives with a Tailwind CSS utility layer.

7.2 Application Entry Point

The Vite entry point (`main.tsx`) renders the `App` component, which wraps the entire application in a provider stack:

```
<ThemeProvider>
  <QueryClientProvider client={queryClient}>
    <AuthProvider>
      <TooltipProvider>
        <BrowserRouter>
          <Routes>
            {/* route definitions */}
          </Routes>
        </BrowserRouter>
      </TooltipProvider>
    </AuthProvider>
  </QueryClientProvider>
</ThemeProvider>
```

Listing 7.1: App.tsx Provider Stack (Simplified)

7.3 Authentication Context

The `AuthContext` provides global authentication state to the entire application. It exposes:

- `user` — the currently authenticated user object (or null).
- `token` — the JWT access token held in memory (not localStorage).
- `login(credentials)` — calls `POST /api/auth/login` and stores the response.
- `logout()` — calls `POST /api/auth/logout` and clears state.
- `isLoading` — true during the initial auth check on mount.

The access token is stored in JavaScript memory (not `localStorage`) to prevent XSS token theft. The refresh token is stored as an HttpOnly cookie, inaccessible to JavaScript.

7.4 Route Guards

Three components implement the route guard hierarchy:

`ProtectedRoute` Checks `AuthContext.user`. If null, redirects to `/login` with a `returnTo` query parameter.

`GuestRoute` The inverse — if the user is already authenticated, redirects them to their role-appropriate dashboard (`/client`, `/freelancer`, or `/admin`).

`RoleRoute` Accepts an `allowedRole` prop (`"CLIENT"`, `"FREELANCER"`, or `"ADMIN"`). If the authenticated user's role does not match, renders a 403 Forbidden screen or redirects.

7.5 Page Inventory

7.5.1 Public Pages

Table 7.1: Public-Facing Pages

Route	Page
`/`	Landing page with hero, feature highlights, and CTA.
`/about`	About page describing the platform and team.
`/how-it-works`	Step-by-step explanation of the Client and Freelancer workflows.
`/privacy`	Privacy policy (legal).
`/terms`	Terms of service (legal).

7.5.2 Authentication Pages

Table 7.2: Authentication Pages

Route	Page
`/signup`	Registration form with email/password + Google OAuth option.
`/login`	Login form with email/password + Google OAuth option.
`/forgot-password`	Email input form to request a password reset link.
`/reset-password`	New password form, reads token from URL query parameter.
`/select-role`	Post-registration role selection (CLIENT or FREELANCER).
`/auth/google/success`	Google OAuth callback page — reads the token from URL and stores it.

7.5.3 Client Pages

Table 7.3: Client Portal Pages

Route	Page / Purpose
`/client`	Dashboard: metrics, recent activity, quick actions.
`/client/post-project`	Multi-step project posting form (8 steps).
`/client/drafts`	List of saved project drafts with resume/delete actions.
`/client/projects`	All published projects with status badges.
`/client/projects/:id`	Project details: description, proposals summary, contract status.
`/client/projects/:id/proposals`	All proposals for a project; shortlist/accept/reject actions.
`/client/browse`	Browse freelancer profiles with filtering sidebar.
`/freelancer/:id`	View a freelancer's public profile from the client side.
`/client/ai-assistant`	AI project scoping chat interface.
`/client/profile`	Edit client profile (company, bio, preferences).
`/client/messages`	Chat room list and messaging interface.
`/client/reviews`	Reviews given and received.
`/client/proposals`	View proposals submitted against client projects.
`/settings`	Account settings (password change, notifications).

7.5.4 Freelancer Pages

Table 7.4: Freelancer Portal Pages

Route	Page / Purpose
`/freelancer`	Dashboard: profile completeness ring, token balance, stats.
`/freelancer/browse`	Browse open projects with filters (category, budget, size, skills).
`/freelancer/proposals`	All submitted proposals with status and token cost.
`/freelancer/projects`	Active and completed contracted projects.
`/freelancer/projects/:id`	Project detail + milestone submission interface.
`/freelancer/projects/:id/proposal`	Proposal submission form (cover letter, price, delivery time).
`/freelancer/profile`	Full profile view and edit.
`/freelancer/tokens`	SkillToken balance, transaction history, token economy info.
`/freelancer/saved`	Bookmarked / saved projects.
`/freelancer/messages`	Chat room list and messaging.
`/freelancer/settings`	Account settings.
Onboarding flow	Multi-step wizard (8 steps).

7.5.5 Admin Pages

Table 7.5: Admin Portal Pages

Route	Page / Purpose
`/admin`	Admin dashboard: platform KPIs, user counts, revenue summary.
`/admin/users`	User management: search, view, ban, unban users.
`/admin/projects`	Project moderation: view and remove flagged projects.
`/admin/disputes`	Dispute management: view, assign, resolve disputes.
`/admin/analytics`	Platform analytics: registration trends, project charts.
`/admin/security`	Security log: admin audit trail and system events.

7.6 Frontend Directory Structure

```
src/
  App.tsx                -- Root component: providers + routing tree
  main.tsx               -- Vite entry point
  contexts/
    AuthContext.tsx      -- Global auth state (user, token, logout)
    ThemeContext.tsx     -- Light/dark theme toggle
  components/
    ProtectedRoute.tsx     -- Auth + role guard components
    layout/                -- Navbar, sidebar, footer
    common/                -- Shared components (cards, badges)
    browse/                -- Browse-specific components
    client/                -- Client-specific components
    modals/                -- Modal dialogs
    ui/                    -- Shadcn UI primitives
  pages/
    LandingPage.tsx
    AboutPage.tsx
    HowItWorksPage.tsx
    auth/                -- Login, Signup, OTP, OAuth, Reset
    client/              -- 14 client-side pages
    freelancer/          -- 12 freelancer-side pages
    admin/               -- 6 admin pages
    legal/               -- Privacy policy, Terms of service
  hooks/                 -- Custom React hooks
  lib/                   -- Utility functions, API client config
```

Listing 7.2: Frontend Directory Structure (ASCII)

7.7 Multi-Step Project Posting Form

The `PostProject.tsx` page is the most complex component in the frontend, with 8 sequential steps validated independently. It uses `React Hook Form` with persistent state across steps via a parent-level form context. Draft saving calls `PATCH /api/projects/:id` with `status: "DRAFT"` on each step transition. The final "Publish" action sends the complete payload with `status: "OPEN"`.

7.8 Freelancer Onboarding Flow

The `Onboarding/` directory contains a step-based wizard that guides new freelancers through profile creation:

1. Basic Info (name, tagline, bio)

2. Skills (skill picker with proficiency levels)
3. Portfolio (image + description + tech stack per item)
4. Certificates (title, organisation, date, URL)
5. Education (school, degree, year)
6. Availability and Preferences (hourly rate, response time, languages)
7. Social Links (GitHub, LinkedIn, portfolio URL)
8. Verification (ID document upload, profile photo upload)

A persistent progress bar shows the completion percentage fetched from the backend after each step.

7.9 Design System and Theming

The frontend implements a design system based on CSS custom properties (design tokens) with two themes: *light* and *dark*. The `ThemeProvider` context toggles a `data-theme` attribute on the `<html>` element, which activates the corresponding token set. Theme preference is persisted in `localStorage` under the key `"skillbridge-theme"`.

The Shadcn UI component library provides a set of accessible, headless primitives (Button, Dialog, Select, Table, Tabs, Toast, etc.) that are styled using the project's Tailwind CSS configuration. All interactive components meet WCAG 2.1 AA accessibility guidelines.

7.10 State Management

Table 7.6: Frontend State Management Strategy

State Type	**Mechanism**
Authentication state	React Context (`AuthContext`) — persisted in memory + HttpOnly cookie for refresh token.
Server state (API data)	TanStack Query — with automatic caching, refetching, and invalidation.
Form state	React Hook Form — with Zod schema validation.
Theme preference	React Context (`ThemeContext`) — persisted in `localStorage`.
UI state (modals, etc.)	Local `useState` within components.

Chapter 8

SkillToken Economy

8.1 Overview

The **SkillToken** is an internal virtual currency native to the SkillBridge platform. It was designed to solve a critical quality problem that plagues open bidding marketplaces: the proliferation of low-effort, mass-submitted proposals. By making proposal submission non-free (but not monetarily costly), the system creates a credible signal of freelancer intent and filters out disengaged bidders.

8.2 Design Rationale

- **Cost without money**: SkillTokens are earned through participation and cannot be purchased with real money. This prevents Pay-to-Win dynamics while still creating scarcity.
- **Reversibility**: Withdrawing a pending proposal refunds the tokens, so a freelancer who changes their mind is not penalised.
- **Progressive cost**: Token cost scales with project budget — larger, higher-value projects cost more tokens to bid on, ensuring freelancers are selective.
- **Reward for activity**: Monthly token awards incentivise platform engagement and profile maintenance.

8.3 Token Events

Table 8.1: SkillToken Ledger Events

Event	Type	Reason Enum	Amount	Trigger
Registration	CREDIT	`REGISTRATION_BONUS`	50	New freelancer account created.
Submit proposal	DEBIT	`PROPOSAL_SUBMITTED`	Varies	Proposal submitted to a project.
Withdraw proposal	CREDIT	`PROPOSAL_WITHDRAWN`	Varies	Pending proposal withdrawn.
Monthly award	CREDIT	`MONTHLY_AWARD`	20	Cron job (last day of month).
Admin grant	CREDIT	`ADMIN_GRANT`	Any	Admin manually grants tokens.
Admin deduct	DEBIT	`ADMIN_DEDUCT`	Any	Admin manually deducts tokens.

8.4 Token Cost Calculation

The `tokenCalculator.ts` utility determines the proposal cost based on the project's budget:

```
export function calculateTokenCost(projectBudget: number): number {
  if (projectBudget < 100)   return 2;
  if (projectBudget < 500)   return 5;
  if (projectBudget < 2000)  return 10;
  if (projectBudget < 10000) return 20;
  return 30; // Premium projects
}
```

Listing 8.1: Token Cost Calculation Logic

This tiered structure ensures that a freelancer proposing on a $50 micro-project pays 2 tokens, while a proposal on a $15,000 enterprise project costs 30 tokens, proportional to the value at stake.

8.5 Token Ledger Model

Every token movement is recorded as an immutable `TokenTransaction` document:

Table 8.2: TokenTransaction Fields

Field	Type	Description
`id`	ObjectId	Primary key.
`freelancerProfileId`	ObjectId	Links to the freelancer's profile.
`type`	TokenWalletTxType	`CREDIT` or `DEBIT`.
`reason`	TokenTxReason	Reason enum (6 values).
`amount`	Int	Always positive; direction determined by `type`.
`balanceAfter`	Int	Running balance snapshot after this transaction.
`description`	String?	Human-readable note (e.g., project title).
`relatedProposalId`	ObjectId?	Linked if reason is proposal-related.
`createdAt`	DateTime	Immutable timestamp.

The `balanceAfter` field enables both fast current-balance lookup (read the most recent transaction) and full historical reconstruction without summing all rows.

8.6 Frontend Token Page

The `FreelancerTokens.tsx` page provides:

- A prominent **balance card** showing the current token balance.
- An **information section** explaining the token economy (how to earn, how to spend, and the cost tiers).
- A **transaction history table** showing all credits and debits, sorted by most recent, with colour-coded amounts (green for credit, red for debit).
- A breakdown of the token cost by proposal value tier.

8.7 Admin Token Controls

Admins can grant or deduct tokens from any freelancer via the User Management panel. This provides an escape hatch for:

- Compensating freelancers affected by platform errors.
- Penalising accounts that abused the system.
- Promotional campaigns.

All admin token operations generate the corresponding `ADMIN_GRANT` or `ADMIN_DEDUCT` `TokenTransaction` and are also logged in `AdminLog` for audit purposes.

Chapter 9

AI Integration

9.1 Overview

SkillBridge integrates the **OpenAI API** [7] to provide an AI-powered project scoping assistant. The assistant is accessible to clients from their dashboard under the route `/client/ai-assistant`. Its primary purpose is to help non-technical clients who know what they want but lack the technical vocabulary to specify it in a form understandable to developers.

9.2 System Design

The AI assistant operates as a **structured-output chat interface**. The client types a natural language description of their project (e.g., *"I need a mobile-friendly website for my bakery that lets customers order cakes online"*), and the assistant returns a structured response that can directly populate a new project posting.

9.2.1 Request Flow

1. The client sends their project description to `POST /api/ai/scope`.
2. The backend `ai.controller.ts` constructs a system prompt and user message, then calls `openai.chat.completions.create()` with the `gpt-4o` model.
3. The OpenAI response is parsed and the structured output is returned as JSON.
4. An `AiScopingSession` document is created in the database storing both the raw input and structured output.
5. If the client proceeds to post the project, the session is linked to the resulting `Project` via `projectId` and the project is flagged `isAiScoped: true` with an `aiScopeSummary` text.

9.2.2 System Prompt Design

The system prompt instructs the model to behave as a freelance project scoping expert and to respond only with a valid JSON object conforming to a specific schema:

```
const systemPrompt = `
You are an expert freelance project scoping assistant.
Given a client's project description, return a JSON object with:
{
  "category": string,          // e.g. "Web Development"
  "subCategory": string,       // e.g. "Frontend"
  "budgetRange": {
    "min": number,
    "max": number
  },
  "projectSize": "SMALL" | "MEDIUM" | "LARGE",
  "skills": string[],          // recommended skills
  "summary": string,           // 2-3 sentence project overview
  "estimatedTimeline": string // e.g. "2-4 weeks"
}
Only respond with the JSON object. No explanations.
`;
```

Listing 9.1: AI System Prompt Structure

9.3 AiScopingSession Model

Table 9.1: AiScopingSession Model Fields

Field	Type	Description
`id`	ObjectId	Primary key.
`userId`	ObjectId	The client who initiated the session.
`input`	String	The raw natural language description provided by the client.
`output`	Json	The structured JSON response from the OpenAI API.
`projectId`	ObjectId?	Linked to the resulting `Project` if the client proceeded to post.
`createdAt`	DateTime	Session timestamp.

9.4 Frontend AI Assistant Page

The `AIAssistant.tsx` page provides a conversational chat-like UI for submitting project descriptions. Key UI elements:

- A large **text area** for the project description.
- A **"Scope My Project"** submit button that triggers the API call.
- While loading: an animated spinner and a motivational message ("Analysing your project...").
- On success: a structured results panel showing the recommended category, budget range, skills, and project size as readable cards.
- A **"Post This Project"** button that pre-fills the `PostProject` multi-step form with the AI-generated data.

9.5 Error Handling and Edge Cases

- If the OpenAI API returns a non-JSON response (e.g., a refusal or safety filter trigger), the controller catches the parse error and returns a 422 status with a user-friendly message.
- API rate limits are handled with exponential back-off retries (up to 3 attempts).
- The OpenAI API key is stored as an environment variable (`OPENAI_API_KEY`) and never exposed to the frontend.

9.6 Use of GPT-4o Model

The `gpt-4o` model was selected for this use case due to its superior instruction-following capabilities and lower latency compared to GPT-4 Turbo. Its ability to reliably produce valid JSON from complex natural-language inputs was a critical selection criterion.

Chapter 10

Security and Authentication

10.1 Overview

Security was a primary design consideration throughout the development of SkillBridge. This chapter documents all authentication mechanisms, authorisation controls, and security-relevant implementation decisions.

10.2 Password Security

All user passwords are hashed using **bcrypt.js** [9] before storage. bcrypt incorporates a configurable work factor (cost factor), which is set to **12** in SkillBridge, making brute-force attacks computationally expensive. The library automatically generates and embeds a cryptographically random salt in each hash, preventing rainbow table attacks.

```
const SALT_ROUNDS = 12;
const passwordHash = await bcrypt.hash(plainTextPassword,
    SALT_ROUNDS);
```

Listing 10.1: Password Hashing During Signup

Plain-text passwords are never stored, logged, or transmitted after the initial POST request body.

10.3 JWT Authentication

SkillBridge uses a **dual-token** JWT authentication scheme:

Table 10.1: JWT Token Properties

Token Type	Lifetime	Storage Location
Access Token	15 minutes	JavaScript memory (React context). Never `localStorage`.
Refresh Token	7 days	HttpOnly cookie (inaccessible to JavaScript; sent automatically by the browser).

Both tokens are signed with separate secrets (`JWT_ACCESS_SECRET` and `JWT_REFRESH_SECRET`) stored as environment variables.

10.3.1 Token Rotation

On each call to `POST /api/auth/refresh`, the backend:

1. Reads the refresh token from the HttpOnly cookie.
2. Looks up the corresponding `Session` document.
3. If found and not expired: deletes the old session, generates a new refresh token, creates a new `Session`, sets a new HttpOnly cookie, and returns a new access token.
4. If the refresh token is reused (already deleted from the DB), a replay attack is detected and all sessions for the user are revoked immediately.

10.4 Google OAuth 2.0

Google authentication is implemented using **Passport.js** [10] with the `passport-google-oauth20` strategy [11]. The flow proceeds as follows:

1. User clicks "Continue with Google" on the frontend.
2. Browser is redirected to `GET /api/auth/google`, which triggers the Passport redirect to Google's authentication servers.
3. User authenticates with Google; Google redirects back to `GET /api/auth/google/callback` with an authorisation code.
4. Passport exchanges the code for a Google profile, using the `googleId` to find or create a `User` document.
5. JWT tokens are generated and the browser is redirected to `/auth/google/success?token=...` with the access token as a URL parameter for a brief handshake (then removed

from URL).

The client credentials (`GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET`) are stored as environment variables only.

10.5 OTP Email Verification

Upon registration, a 6-digit OTP is generated using `crypto.randomInt(100000, 999999)`. It is:

- Stored in Redis with a 10-minute TTL under the key `otp:{email}`.
- Sent via Nodemailer to the user's email as an HTML-formatted message.
- Deleted from Redis upon successful verification.

Failed OTP attempts are not currently rate-limited (planned future enhancement), but the short TTL limits the attack window.

10.6 Role-Based Access Control (RBAC)

All protected API routes are guarded by two layers of middleware:

1. **`protect`** — verifies the JWT access token and attaches the decoded `{userId, role}` to `req.user`.
2. **`requireRole(role)`** — checks that `req.user.role === role`; returns 403 if not.

The frontend enforces the same constraints via `RoleRoute` components that render a 403 screen or redirect if the user's role does not match.

10.7 Environment Variable Security

All secrets and configuration values are stored in a `.env` file:

- `.env` is listed in `.gitignore` to prevent accidental commitment.
- Environment variables are accessed exclusively via `process.env.*` after calling `dotenv.config()`.
- The `.env` file is never exposed to the frontend (Vite/React receives only `VITE_*` prefixed variables, and none of the backend secrets carry this prefix).

See Appendix B for a full list of required environment variables.

10.8 CORS Configuration

Cross-Origin Resource Sharing is configured to allow requests only from the specific frontend origins. The current development configuration allows:

- `http://localhost:5173` (Vite dev server)
- `http://localhost:3000`
- `http://localhost:8080`

`credentials: true` is enabled to allow the browser to send HttpOnly cookies with cross-origin requests.

10.9 File Upload Security

- Multer enforces a **10MB** maximum file size.
- Files are stored in Cloudinary, not on the application server. The server never stores files on its filesystem.
- Uploaded file types are not explicitly validated in the current version (planned future enhancement).

10.10 XSS and Injection Prevention

- The access token is stored in JavaScript memory, not `localStorage`, preventing XSS token theft.
- All request body data is validated via **Zod** schemas before database operations.
- Prisma's parameterised query model prevents SQL/NoSQL injection at the ORM level.

10.11 Account Security Features

- **Account ban**: Admins can ban accounts. Banned users receive a 403 response on login with the ban reason.
- **ID verification**: A boolean `isIdVerified` field is set by admins after reviewing the uploaded ID document.
- **Password reset**: A UUID-based reset token (stored hashed in `PasswordResetToken`)

with a 1-hour expiry. Tokens are single-use (marked `used: true` on consumption).

Chapter 11

Testing and Validation

11.1 Overview

This chapter describes the testing strategy employed during the development of SkillBridge. Due to the nature of this project as an academic prototype, the primary focus is on **integration-level testing** of the API, **functional validation** of the frontend, and **data seeding** for end-to-end scenario verification.

11.2 Testing Strategy

Table 11.1: Testing Strategy Overview

Level	Method	Scope
API Integration	Manual (Postman / HTTP client)	All REST endpoints and auth flows.
Data Seeding	TypeScript seed scripts	Database population for test scenarios.
Frontend	Manual browser walkthrough	All page flows, role-specific routes, form validation.
Schema	Prisma type checking	All model relationships and field types verified at compile time.

11.3 API Testing

Each API endpoint was tested manually using an HTTP client (Postman / cURL) during development. The following categories of tests were performed:

11.3.1 Authentication Flow Tests

Table 11.2: Authentication API Test Cases

Test Case	Description	Result
TC-AUTH-01	Register with valid email and password; OTP sent to email.	Pass
TC-AUTH-02	Register with duplicate email; expect 409 error.	Pass
TC-AUTH-03	Verify OTP with correct code within 10 min; access token returned.	Pass
TC-AUTH-04	Verify OTP with expired code; expect 400 error.	Pass
TC-AUTH-05	Login with correct credentials; JWT tokens issued.	Pass
TC-AUTH-06	Login with wrong password; expect 401 error.	Pass
TC-AUTH-07	Login with banned account; expect 403 with ban reason.	Pass
TC-AUTH-08	Refresh token endpoint; new access token issued; old cookie replaced.	Pass
TC-AUTH-09	Logout; session deleted; next refresh fails.	Pass
TC-AUTH-10	Password reset flow: request email â†' receive link â†' set new password.	Pass

11.3.2 Project and Proposal Tests

Table 11.3: Project and Proposal Test Cases

Test Case	Description	Result
TC-PROJ-01	Create project as CLIENT role; status = DRAFT.	Pass
TC-PROJ-02	Attempt project creation as FREELANCER role; expect 403.	Pass
TC-PROJ-03	Publish project (DRAFT â†' OPEN).	Pass
TC-PROP-01	Submit proposal as FREELANCER; token deducted; proposal created.	Pass
TC-PROP-02	Submit proposal with insufficient balance; expect 400 error.	Pass
TC-PROP-03	Submit duplicate proposal (same project); expect 409 error.	Pass
TC-PROP-04	Withdraw pending proposal; tokens refunded.	Pass
TC-PROP-05	Accept proposal as CLIENT; Contract created; project = IN_PROGRESS.	Pass

11.3.3 Token Economy Tests

Table 11.4: SkillToken Economy Test Cases

Test Case	Description	Result
TC-TOK-01	New freelancer receives registration bonus of 50 tokens.	Pass
TC-TOK-02	Token transaction ledger records correct `balanceAfter`.	Pass
TC-TOK-03	Admin grants tokens to freelancer; balance and ledger updated.	Pass
TC-TOK-04	Admin deducts tokens from freelancer; balance and ledger updated.	Pass

11.4 Data Seeding

The `prisma/` directory contains a suite of TypeScript seed scripts used to populate the database with realistic test data:

Table 11.5: Database Seed Scripts

File	Content
`seed.ts`	Master seed orchestrator; calls all other seed functions in correct order.
`seed_metadata.ts`	Populates the skills catalogue, categories, subcategories, languages, and locations.
`seedFreelancers.ts`	Creates 50 realistic freelancer accounts with complete profiles, skills, portfolio items, and token balances.
`seedClients.ts`	Creates 20 client accounts with company profiles and posted projects in various statuses.

Seeds are executed with:

```
npm run seed
# or
npx tsx prisma/seed.ts
```

Listing 11.1: Running the Database Seed

The seed data provides a fully populated database for development and demonstration purposes, including freelancers with varying SkillToken balances, clients with open projects, proposals in multiple states, and active contracts with milestones.

11.5 Frontend Validation Testing

All frontend pages were manually tested in both light and dark themes across viewport widths of 375px (mobile), 768px (tablet), and 1920px (desktop). The following key flows were validated:

1. **Full registration-to-dashboard flow**: Signup â†' OTP verification â†' Role selection â†' Dashboard.
2. **Freelancer onboarding**: All 8 steps of the onboarding wizard, including image uploads and draft saving.
3. **Project posting**: All 8 steps of the PostProject form, including draft save and publish.
4. **Proposal submission**: Browse projects â†' View project â†' Submit proposal â†' Verify token deducted.
5. **Contract workflow**: Accept proposal â†' View contract â†' Submit milestone â†' Approve milestone.
6. **AI scoping**: Enter project description â†' Receive structured output â†' Pre-fill PostProject form.
7. **Admin flows**: Login as admin â†' Ban user â†' Resolve dispute â†' Grant tokens.

11.6 TypeScript Type Safety

TypeScript's static type system provided a continuous layer of correctness checking throughout development. The Prisma-generated client types ensured that all database queries and mutations were type-correct at compile time, preventing entire categories of runtime errors that are common in untyped JavaScript backends. Running `npx tsc -noEmit` reported zero type errors in the final submission codebase.

11.7 Known Limitations

- No automated unit tests or integration tests are implemented (planned future work).
- Payment gateway integration is simulated via status fields; no real money flows are tested.
- Real-time messaging (Socket.IO) is architecturally planned but not fully implemented in the current version â€" chat rooms and messages are persisted but live

push is handled via polling.

- OTP rate-limiting is not yet implemented, leaving the OTP endpoint theoretically vulnerable to brute-force within the 10-minute window.

Chapter 12

Conclusion and Future Work

12.1 Summary

This report has presented the complete design, architecture, and implementation of **SkillBridge**, a full-stack freelance marketplace platform developed as a Final Year Project. The system successfully demonstrates the end-to-end development of a production-grade web application encompassing:

- A secure, multi-role **REST API** built with Node.js, Express, and TypeScript, backed by MongoDB via the Prisma ORM.
- A **React single-page application** with role-based routing, context-based state management, and a polished design system.
- A novel **SkillToken economy** that reduces low-quality proposal spam through a token-based bidding cost mechanism.
- An **AI project-scoping assistant** that leverages OpenAI's GPT-4o to translate informal project descriptions into structured specifications.
- A **blind peer-review system** ensuring rating integrity between clients and freelancers.
- A complete **contract and milestone management** workflow with payment tracking.
- An **admin moderation system** with dispute resolution, user management, and audit logging.

All four research gaps identified in Chapter 2 — AI scoping, token economy, blind reviews, and open-architecture reference implementation — have been addressed in the delivered system.

12.2 Achievements

Table 12.1: Project Objectives vs. Achievements

Objective	Description	Status
OBJ-01	Secure multi-role REST API	**Achieved**
OBJ-02	Responsive role-aware SPA	**Achieved**
OBJ-03	SkillToken economy with ledger	**Achieved**
OBJ-04	OpenAI AI scoping assistant	**Achieved**
OBJ-05	Blind review system	**Achieved**
OBJ-06	Role-Based Access Control (RBAC)	**Achieved**
OBJ-07	Multi-step freelancer onboarding	**Achieved**

12.3 Limitations

While all primary objectives were achieved, the following limitations are acknowledged:

1. **No automated test suite**: The absence of unit and integration tests means that regression testing is entirely manual, which is not scalable.
2. **Payment simulation**: The payment gateway is simulated through status fields. Integration with a real gateway (e.g., Stripe) was beyond scope.
3. **Real-time messaging**: While the data model supports real-time chat via Socket.IO, the current implementation uses HTTP polling for message retrieval.
4. **No mobile application**: The platform is web-only. A native mobile experience would significantly expand the user base.
5. **Limited OTP security**: The OTP endpoint currently lacks rate-limiting, which could be exploited for brute-force attacks within the 10-minute window.
6. **Deployment**: The system is documented as a local development setup. Production deployment (containerisation, CI/CD, cloud hosting) is beyond the scope of this academic submission.

12.4 Future Work

Based on the current implementation and identified limitations, the following enhancements are proposed for future iterations:

12.4.1 Technical Enhancements

1. **Automated testing**: Implement a full Jest-based test suite with unit tests for utility functions and integration tests for all API endpoints.
2. **Real-time messaging**: Replace HTTP polling with a Socket.IO WebSocket integration for genuinely real-time chat.
3. **Payment gateway**: Integrate Stripe for actual escrow and release of funds, replacing the current status-based simulation.
4. **OTP rate-limiting**: Implement Redis-based rate-limiting (max 5 attempts per 10-minute window) for the OTP verification endpoint.
5. **CI/CD pipeline**: Set up GitHub Actions for automated linting, type-checking, testing, and deployment.
6. **Docker containerisation**: Package all services (API, Redis) in Docker containers with a `docker-compose.yml` for one-command setup.

12.4.2 Feature Enhancements

1. **Video calling**: Integrate WebRTC for in-platform client-freelancer video consultations.
2. **Smart recommendations**: Use machine learning to recommend projects to freelancers and freelancers to clients based on interaction history.
3. **Subscription tiers**: Introduce premium freelancer tiers with higher monthly token allotments and profile badges.
4. **Mobile application**: Develop React Native iOS and Android apps sharing the same backend API.
5. **Skill assessments**: Introduce platform-administered skill tests whose results are displayed as verified badges on freelancer profiles.
6. **Analytics dashboard**: Expand the admin dashboard with real-time platform analytics, revenue reporting, and geographic distribution maps.

12.5 Conclusion

SkillBridge demonstrates that a feature-rich, production-quality freelance marketplace can be designed and implemented within an academic project scope using a modern, open-source technology stack. The system's distinguishing features — the SkillToken

economy, AI-assisted project scoping, and blind review system — represent genuine innovations over existing commercial platforms that could be further developed into a viable product.

The development of this project provided invaluable practical experience in full-stack web development, software architecture, database design, API security, and the integration of third-party AI services. The system serves as a comprehensive reference implementation for future students and researchers interested in marketplace platform design.

Bibliography

[1] Upwork Inc. Upwork – the world's work marketplace. https://www.upwork.com, 2024. Accessed: March 2025.

[2] Fiverr International Ltd. Fiverr – freelance services marketplace. https://www.fiverr.com, 2024. Accessed: March 2025.

[3] Freelancer.com Limited. Freelancer – hire and find jobs online. https://www.freelancer.com, 2024. Accessed: March 2025.

[4] Toptal LLC. Toptal – hire expert software developers. https://www.toptal.com, 2024. Accessed: March 2025.

[5] Sarah Kessler. The gig economy and the future of work. *Journal of Economic Perspectives*, 37(1):45–62, 2023.

[6] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, Boston, MA, 1994.

[7] OpenAI. Openai api documentation. https://platform.openai.com/docs, 2024. Accessed: March 2025.

[8] Roy T. Fielding. Architectural styles and the design of network-based software architectures. *University of California, Irvine – Doctoral Dissertation*, 2000.

[9] dcode. bcrypt.js – optimised bcrypt in javascript. https://github.com/dcodeIO/bcrypt.js. Accessed: March 2025.

[10] Jared Hanson. Passport.js – simple, unobtrusive authentication for node.js. https://www.passportjs.org. Accessed: March 2025.

[11] D. Hardt. The OAuth 2.0 Authorization Framework – RFC 6749. https://tools.ietf.org/html/rfc6749, 2012. Internet Engineering Task Force (IETF).

Appendix A

API Endpoint Reference

This appendix documents all REST API endpoints exposed by the SkillBridge backend. Protected routes require a `Bearer` JWT access token in the `Authorization` header unless stated otherwise.

Legend

- [**PUB**] â€” Public (no authentication required)
- [**AUTH**] â€” Requires authentication (any role)
- [**CLIENT**] â€” Requires CLIENT role
- [**FRLNC**] â€— Requires FREELANCER role
- [**ADMIN**] â€— Requires ADMIN role

Authentication â€— `/api/auth`

Method	Endpoint	Access	Description
POST	`/api/auth/signup`	PUB	Register new user account; triggers OTP email.
POST	`/api/auth/verify-otp`	PUB	Verify email with OTP code.
POST	`/api/auth/login`	PUB	Login with email & password; returns JWT tokens.
POST	`/api/auth/refresh`	PUB	Rotate refresh token; return new access token.
POST	`/api/auth/logout`	AUTH	Invalidate current session.
POST	`/api/auth/forgot-password`	PUB	Send password reset email link.
POST	`/api/auth/reset-password`	PUB	Reset password using the emailed token.
GET	`/api/auth/google`	PUB	Initiate Google OAuth 2.0 flow.

Method	Endpoint	Access	Description
GET	`/api/auth/google/callback`	PUB	Google OAuth 2.0 callback handler.
POST	`/api/auth/complete-google-signup`	AUTH	Set role after Google sign-in.

Projects â€” /api/projects

Method	Endpoint	Access	Description
GET	`/api/projects`	PUB	Get all open projects (paginated, filtered).
POST	`/api/projects`	CLIENT	Create a new project (status: DRAFT).
GET	`/api/projects/:id`	PUB	Get project by ID.
PATCH	`/api/projects/:id`	CLIENT	Update project details.
DELETE	`/api/projects/:id`	CLIENT	Delete a project.
PATCH	`/api/projects/:id/publish`	CLIENT	Transition project DRAFT â†' OPEN.
PATCH	`/api/projects/:id/status`	CLIENT	Update project status (e.g., COMPLETED).

Proposals â€” /api/proposals

Method	Endpoint	Access	Description
POST	`/api/proposals`	FRLNC	Submit proposal; deducts SkillTokens.
GET	`/api/proposals/my`	FRLNC	Get all proposals submitted by current freelancer.
GET	`/api/proposals/project/:projectId`	CLIENT	Get all proposals for a specific project.
PATCH	`/api/proposals/:id/withdraw`	FRLNC	Withdraw pending proposal; refunds tokens.
PATCH	`/api/proposals/:id/shortlist`	CLIENT	Mark proposal as shortlisted.

Method	Endpoint	Access	Description
PATCH	`/api/proposals/:id/accept`	CLIENT	Accept proposal; creates Contract.
PATCH	`/api/proposals/:id/reject`	CLIENT	Reject proposal.

Freelancers â€” `/api/freelancers`

Method	Endpoint	Access	Description
GET	`/api/freelancers/profile`	FRLNC	Get own freelancer profile.
PATCH	`/api/freelancers/onboarding/basic-info`	FRLNC	Update basic info (step 1).
PATCH	`/api/freelancers/onboarding/skills`	FRLNC	Update skills (step 2).
PATCH	`/api/freelancers/onboarding/portfolio`	FRLNC	Update portfolio items (step 3).
PATCH	`/api/freelancers/onboarding/certificates`	FRLNC	Update certificates (step 4).
PATCH	`/api/freelancers/onboarding/education`	FRLNC	Update education (step 5).
PATCH	`/api/freelancers/onboarding/availability`	FRLNC	Update availability & rates (step 6).
PATCH	`/api/freelancers/onboarding/social-links`	FRLNC	Update social links (step 7).
PATCH	`/api/freelancers/onboarding/verification`	FRLNC	Upload ID document (step 8).
GET	`/api/freelancers/:id`	AUTH	Get freelancer public profile by ID.

Tokens â€” `/api/tokens`

Method	Endpoint	Access	Description
GET	`/api/tokens/balance`	FRLNC	Get current SkillToken balance.
GET	`/api/tokens/history`	FRLNC	Get full token transaction history.

Method	Endpoint	Access	Description
POST	`/api/tokens/admin/grant`	ADMIN	Grant tokens to a freelancer.
POST	`/api/tokens/admin/deduct`	ADMIN	Deduct tokens from a freelancer.

Browse â€” `/api/browse`

Method	Endpoint	Access	Description
GET	`/api/browse/freelancers`	AUTH	Browse freelancer profiles with filters and scoring.
GET	`/api/browse/projects`	AUTH	Browse projects with filters.

Tracking â€” `/api/track`

Method	Endpoint	Access	Description
POST	`/api/track/view/:projectId`	FRLNC	Record a project view interaction.
POST	`/api/track/save/:projectId`	FRLNC	Save/bookmark a project.
DELETE	`/api/track/save/:projectId`	FRLNC	Remove a project bookmark.
GET	`/api/track/saved`	FRLNC	Get all saved/bookmarked projects.

Other Endpoints

Method	Endpoint	Access	Description
GET	`/api/categories`	PUB	List all categories with subcategories.
GET	`/api/metadata/languages`	PUB	List available languages.
GET	`/api/metadata/locations`	PUB	List available locations.
POST	`/api/ai/scope`	CLIENT	Submit a project description to the AI scoping assistant.

Appendix B

Environment Variables Reference

The SkillBridge backend is configured entirely through environment variables loaded from a `.env` file in the project root. **This file must never be committed to version control.** The following table documents all required and optional variables.

Variable	Required	Description
Server		
`PORT`	Optional	HTTP server port. Default: `5000`.
`NODE_ENV`	Optional	Environment mode. `development` or `production`.
Database		
`DATABASE_URL`	Required	Full MongoDB connection string (Atlas or local). Example: `mongodb+srv://user:pass@cluster.mongodb`
JWT		
`JWT_ACCESS_SECRET`	Required	Secret key for signing access tokens. Use a long, random string.
`JWT_REFRESH_SECRET`	Required	Secret key for signing refresh tokens. Must be different from access secret.
Google OAuth		
`GOOGLE_CLIENT_ID`	Required	Google OAuth 2.0 client ID from Google Cloud Console.
`GOOGLE_CLIENT_SECRET`	Required	Google OAuth 2.0 client secret.
`GOOGLE_CALLBACK_URL`	Required	OAuth redirect URI. Example: `http://localhost:5000/api/auth/google/c`
Redis		
`REDIS_URL`	Required	Redis connection URL. Example: `redis://localhost:6379`.

Variable	Required	Description
Cloudinary		
`CLOUDINARY_CLOUD_NAME`	Required	Cloudinary account cloud name.
`CLOUDINARY_API_KEY`	Required	Cloudinary API key.
`CLOUDINARY_API_SECRET`	Required	Cloudinary API secret.
Nodemailer (SMTP)		
`SMTP_HOST`	Required	SMTP server hostname. Example: `smtp.gmail.com`.
`SMTP_PORT`	Required	SMTP port. Typically `587` (TLS) or `465` (SSL).
`SMTP_USER`	Required	SMTP authentication email address.
`SMTP_PASS`	Required	SMTP authentication password or App Password.
`EMAIL_FROM`	Required	Sender display address. Example: `"SkillBridge" <no-reply@skillbridge.com>`.
OpenAI		
`OPENAI_API_KEY`	Required	OpenAI API secret key (starts with `sk-`).
Frontend		
`FRONTEND_URL`	Required	Base URL of the frontend app. Example: `http://localhost:5173`. Used for CORS and OAuth redirect fallback URLs.

Sample `.env` File

```
# Server
PORT=5000
NODE_ENV=development

# Database (MongoDB Atlas)
DATABASE_URL="mongodb+srv://username:password@cluster.mongodb.net/
    skillbridge"

# JWT
```

```
JWT_ACCESS_SECRET="your-very-long-random-access-secret-here"
JWT_REFRESH_SECRET="another-very-long-random-refresh-secret-here"

# Google OAuth
GOOGLE_CLIENT_ID="123456789-abc.apps.googleusercontent.com"
GOOGLE_CLIENT_SECRET="GOCSPX-your-client-secret"
GOOGLE_CALLBACK_URL="http://localhost:5000/api/auth/google/callback
    "

# Redis
REDIS_URL="redis://localhost:6379"

# Cloudinary
CLOUDINARY_CLOUD_NAME="your-cloud-name"
CLOUDINARY_API_KEY="123456789012345"
CLOUDINARY_API_SECRET="your-api-secret"

# Email (SMTP)
SMTP_HOST="smtp.gmail.com"
SMTP_PORT=587
SMTP_USER="your-email@gmail.com"
SMTP_PASS="your-16-char-app-password"
EMAIL_FROM='"SkillBridge" <no-reply@skillbridge.com>'

# OpenAI
OPENAI_API_KEY="sk-proj-your-openai-key-here"

# Frontend
FRONTEND_URL="http://localhost:5173"
```

Listing B.1: Sample .env Configuration

Appendix C

Database Schema Quick Reference

This appendix provides a concise tabular reference for every model in the SkillBridge Prisma schema (`prisma/schema.prisma`).

Table C.1: All Database Models Summary

Model	**Collection**	**Primary Purpose**
User	users	Unified auth entity for all roles.
Session	sessions	Refresh token server-side storage.
ClientProfile	client_profiles	Client-specific profile data.
FreelancerProfile	freelancer_profiles	Freelancer-specific profile data.
AdminProfile	admin_profiles	Admin account metadata.
Project	projects	Client project listings.
ProjectSkill	project_skills	Many-to-many: Project ↔ Skill.
Proposal	proposals	Freelancer bids on projects.
Contract	contracts	Active agreements from accepted proposals.
Milestone	milestones	Deliverable checkpoints within contracts.
Payment	payments	Payment/escrow status per milestone.
Dispute	disputes	Contract dispute records.
Review	reviews	Blind post-contract reviews.
ChatRoom	chat_rooms	Messaging rooms per contract or pre-contract.
Message	messages	Individual chat messages.
Notification	notifications	In-app notification feed.
Skill	skills	Global skills catalogue.
FreelancerSkill	freelancer_skills	Many-to-many: Freelancer ↔ Skill.
PortfolioItem	portfolio_items	Freelancer past work showcase.
Certificate	certificates	Freelancer professional credentials.
Education	education	Freelancer education history.
Gig	gigs	Freelancer pre-packaged services.
Category	categories	Project category taxonomy.
SubCategory	sub_categories	Project sub-category taxonomy.
Invitation	invitations	Client-to-freelancer project invitations.
TokenTransaction	token_transactions	Immutable SkillToken ledger.
AiScopingSession	ai_scoping_sessions	Stored AI assistant conversations.
SavedProject	saved_projects	Freelancer project bookmarks.
BrowseInteraction	browse_interactions	Browsing analytics for personalisation.
AdminLog	admin_logs	Immutable admin action audit trail.
PlatformSetting	platform_settings	Global key-value configuration.
PasswordResetToken	(embedded)	Secure password reset tokens.

Key Relationships Diagram (Text)

```
User (1)--(1) ClientProfile
     (1)--(1) FreelancerProfile
     (1)--(1) AdminProfile
     (1)--(*) Session
     (1)--(*) Notification

ClientProfile (1)--(*) Project
              (1)--(*) Invitation

FreelancerProfile (1)--(*) Proposal
                  (1)--(*) FreelancerSkill--(*) Skill
                  (1)--(*) PortfolioItem
                  (1)--(*) Certificate
                  (1)--(*) Education
                  (1)--(*) Gig
                  (1)--(*) TokenTransaction

Project (1)--(*) Proposal
        (1)--(1) Contract
        (1)--(*) ProjectSkill--(*) Skill
        (1)--(*) Invitation

Contract (1)--(*) Milestone
         (1)--(*) Payment
         (1)--(1) ChatRoom
         (1)--(1) Dispute

Milestone (1)--(1) Payment

ChatRoom (1)--(*) Message
```

Listing C.1: Core Model Relationship Summary